

Formation RAG

Retrieval-Augmented Generation

Séance 1 | Fondements théoriques

David Christ MEKONTCHOU Nzone

Analyste statisticien – Data Scientist

Mai 2026





Au programme

Introduction

Fondations

Les LLM

Les limites des LLM

Positionnement du RAG

Qu'est-ce qu'un RAG ?

Fonctionnement détaillé

Composants techniques

Les types de RAG

Évaluation

Avantages, limites, pièges

Cas d'usage

Écosystème & outils

Bridge vers la pratique

Introduction

Pourquoi cette formation ?

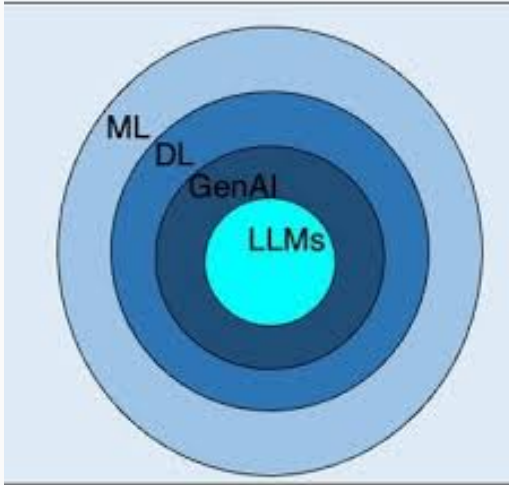
- Les LLM ont explosé depuis 2022 (ChatGPT, Claude, Gemini...)
- Mais ils ont des **limites structurelles** pour un usage en entreprise
- Le **RAG** est aujourd'hui l'une des solutions les plus déployées
- Notre objectif sur 3 séances :
 1. Comprendre (aujourd'hui)
 2. Construire un RAG simple
 3. Construire un RAG robuste

Plan de la séance 1

1. IA, ML, Deep Learning, GenAI
2. Les LLM
3. Les limites des LLM
4. Positionnement du RAG
5. Qu'est-ce qu'un RAG ?
6. Fonctionnement détaillé
7. Composants techniques
8. Les types de RAG
9. L'évaluation
10. Avantages, limites, pièges
11. Cas d'usage
12. Écosystème & outils
13. Bridge vers la pratique

Fondations

IA, ML, DL, GenAI : où on est ?



- **IA** : tout système intelligent
- **ML** : l'IA apprend depuis des données
- **DL** : ML avec des réseaux de neurones profonds
- **GenAI** : produit du *nouveau* contenu
- **LLM** : la GenAI appliquée au texte

- Idée centrale : au lieu de coder des règles, on *apprend* depuis des exemples
- Trois familles :
 - **Supervisé** (avec étiquettes)
 - **Non supervisé** (sans étiquettes)
 - **Renforcement** (par récompense)

Exemple

Détection de spam : on ne liste pas les mots interdits, on montre des milliers d'exemples et le modèle apprend seul.

Deep Learning : réseaux de neurones à plusieurs couches.

Trois facteurs ont fait exploser le domaine depuis 2012 :

- Données massives
- Puissance de calcul (GPU)
- Algorithmes (backpropagation, ReLU...)

IA générative : classe de modèles qui **produisent du nouveau contenu**.

- Texte : GPT-4, Claude, Llama, Mistral
- Image : DALL-E, Midjourney, Stable Diffusion
- Audio, vidéo, code...

Les LLM

Qu'est-ce qu'un LLM ?

Définition

Un **Large Language Model** est un modèle de Deep Learning entraîné sur des milliers de milliards de mots, capable de générer du langage naturel en prédisant le *token suivant* le plus probable.

À retenir

Intuition centrale : un LLM est une **machine à compléter du texte**.

Année	Jalon
2013	Word2Vec — premiers embeddings de mots
2017	Transformer — <i>Attention Is All You Need</i>
2018	BERT, GPT-1
2020	GPT-3 — 175B paramètres
2022	ChatGPT — démocratisation grand public
2023+	GPT-4, Claude, Gemini, Llama, Mistral
2025+	Modèles raisonneurs, contextes 1M+ tokens

Trois notions à connaître absolument

- **Token** : unité de base manipulée par le modèle (sous-mot).
1 token $\approx$ 4 caractères en anglais.
- **Embedding** : représentation vectorielle (liste de nombres) d'un texte.
Sémantique proche $\Rightarrow$ vecteurs proches.
- **Fenêtre de contexte** : nombre max de tokens en entrée du LLM.
GPT-4o : 128k. Claude 4 : 200k–1M. Gemini : 2M.

Les embeddings : l'idée à retenir



Point essentiel

Cette idée est **centrale** pour le RAG.

Tout ce que nous ferons ensuite repose dessus.

Que savent faire les LLM ?

- Générer du texte
- Résumer
- Traduire
- Classer / extraire
- Question / Réponse
- Coder
- Dialoguer
- Reasonner (de mieux en mieux)



Les limites des LLM

Les LLM ont 6 limites majeures

1. **Hallucinations**
2. **Knowledge cutoff** (date de coupure)
3. **Pas d'accès aux données privées**
4. **Pas de sources / traçabilité**
5. **Fenêtre de contexte limitée**
6. **Coût et lourdeur du fine-tuning**

À retenir

Le RAG est conçu pour répondre à ces limites.

Définition

Hallucination : génération d'une information plausible mais factuellement fausse, présentée avec assurance.

Exemple

Q : "Qui a gagné le Nobel de physique 2025 ?"

R : "David Christ, pour ses travaux sur la fusion froide."

Le modèle invente parce qu'il ne sait pas.

- **Knowledge cutoff** : le modèle ignore tout événement postérieur à sa date d'entraînement.
- **Données privées** : il n'a jamais lu vos documents internes, contrats, tickets.
- **Pas de sources** : aucune traçabilité. Bloquant en juridique, médical, audit.

- **Fenêtre de contexte** : même 1M tokens ne suffit pas pour toute la doc d'une entreprise, et plus on remplit, plus c'est cher et lent.
- **Fine-tuning** : coûteux, peu réactif, sans sources, risque de *catastrophic forgetting*.

Synthèse : le RAG répond à toutes ces limites

Limite	Problème	RAG résout ?
Hallucinations	Info fausse	Oui (ancrage)
Knowledge cutoff	Pas d'actu	Oui
Données privées	Inaccessibles	Oui
Sources	Aucune	Oui
Contexte limité	Coût/qualité	Oui (top-k ciblé)
Fine-tuning lourd	Réactivité	Oui (plus léger)

Positionnement du RAG

Trois approches pour adapter un LLM

- **Prompt Engineering** : bien formuler la requête.
- **Fine-tuning** : réentraîner le modèle sur ses données.
- **RAG** : récupérer dynamiquement les documents pertinents.

À retenir

Ces trois approches **ne s'excluent pas**. En production, on les combine.

Matrice de décision

Critère	Prompt Eng.	Fine-tuning	RAG
Coût initial	Très faible	Élevé	Moyen
Mise à jour	Manuelle	Réentraînement	Auto
Sources	Non	Non	Oui
Données privées	Limité	Oui	Oui
Maîtrise du style	Limitée	Excellente	Limitée
Complexité	Faible	Élevée	Moyenne

Qu'est-ce qu'un RAG ?

Définition

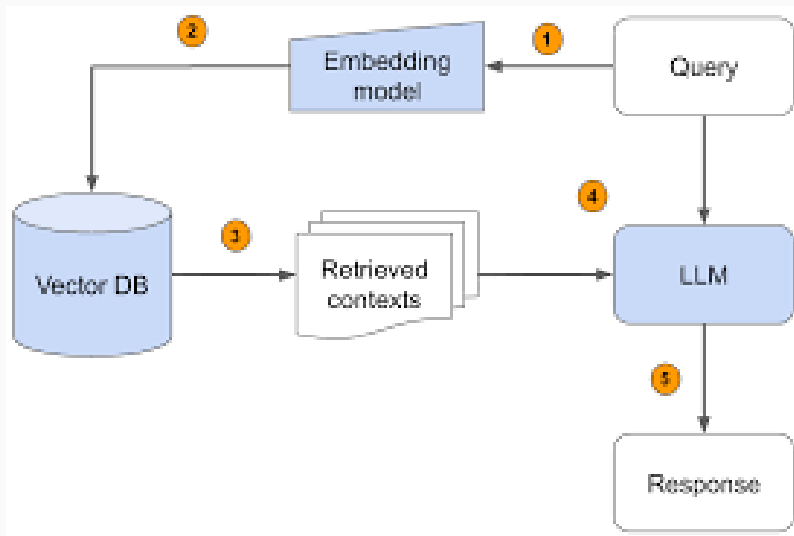
Un système **RAG** (*Retrieval-Augmented Generation*) recherche dans une base documentaire les passages pertinents pour une question, les fournit au LLM comme contexte, puis lui demande de générer la réponse.

À retenir

Analogie : un examen à livre ouvert pour le LLM.

- Papier fondateur : **Lewis et al., 2020** (Facebook AI Research)
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
- Idée : coupler un *retriever* (moteur de recherche) à un *generator* (LLM)
- Boom à partir de 2023 avec ChatGPT et la maturation des vector stores

Schéma général d'un RAG



Ce qu'un RAG n'est PAS

- Ce **n'est pas** un moteur de recherche classique (il *génère* une réponse)
- Ce **n'est pas** un fine-tuning (le modèle n'est pas modifié)
- Ce **n'est pas** une garantie magique contre les hallucinations
Si la base est mauvaise, le LLM peut toujours inventer.

Fonctionnement détaillé

Deux phases bien distinctes

1. **Indexation** (*offline*)

Prépare la base de connaissances. Une seule fois (puis à chaque MAJ).

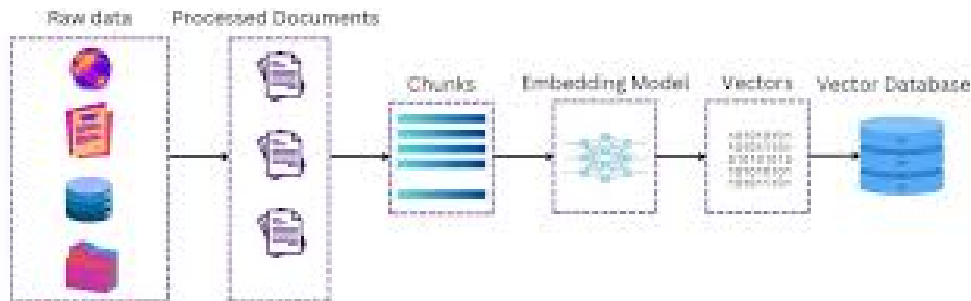
2. **Retrieval + Generation** (*online*)

Exécuté à chaque question utilisateur.

TechnoHealth Solutions' RAG Implementation



Step 1: Building an Indexing Pipeline



Étape 1.1 — Charger les documents

On extrait le texte (et la structure) de sources variées :

- PDF, Word, HTML, Markdown
- Bases de données SQL
- APIs (Notion, Confluence, Slack, Drive)
- Audio (Whisper), images (OCR)

Chaque format $\Rightarrow$ un *loader* dédié.

Étape 1.2 — Le chunking (découpage)

Définition

Découper chaque document en passages (*chunks*) de quelques centaines à quelques milliers de caractères.

Pourquoi ?

- Précision de la recherche
- Respect de la fenêtre de contexte
- Qualité des embeddings (un vecteur représente mal un livre entier)

- **Fixed-size** : nombre fixe de caractères. Le plus simple.
- **Recursive** : respecte paragraphes/phrases. *Par défaut.*
- **Semantic** : frontières détectées par un modèle.
- **Document-aware** : respecte la structure (titres).

Attention

Chunk overlap : chevauchement de quelques tokens entre chunks consécutifs, indispensable pour ne pas perdre les informations à la frontière.

Étape 1.3 — Vectorisation (embeddings)

Chaque chunk est transformé en vecteur via un *modèle d'embedding*.

Exemple

"Le RAG combine recherche et génération."
→ [0.12, -0.05, 0.88, ...] (1536 nombres)

Modèles courants :

- OpenAI text-embedding-3, Cohere embed-v3
- Open-source : BGE, E5, Sentence-Transformers (MiniLM)

Étape 1.4 — Stocker dans un vector store

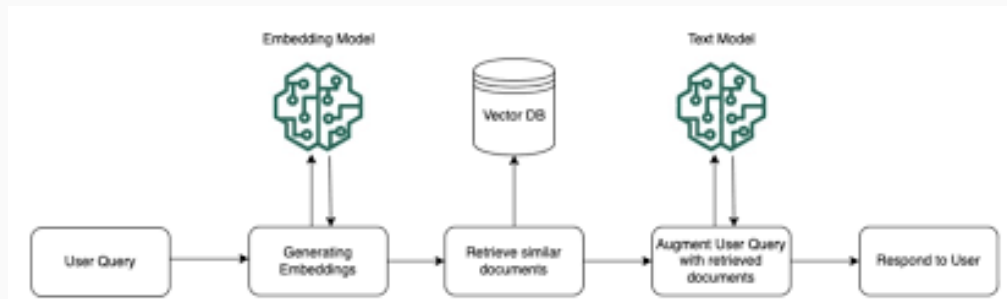
Définition

Vector store : base de données spécialisée qui stocke des millions de vecteurs et y effectue des recherches de similarité ultra-rapides (algorithmes ANN).

On y enregistre :

- le vecteur
- le texte original du chunk
- des métadonnées (source, page, date, auteur...)

Phase 2 — Retrieval + Generation



Étapes 2.1 et 2.2 — Embedder et rechercher

1. La question est embeddée avec le **même modèle** qu'à l'indexation.
2. Le vector store renvoie les **top-k** chunks les plus proches.
Mesures : similarité cosinus, distance euclidienne, produit scalaire.

Attention

Utiliser un modèle d'embedding différent entre indexation et requête $\Rightarrow$ résultats catastrophiques.

Étape 2.3 — Construire le prompt augmenté

Tu es un assistant qui répond aux questions en t'appuyant **uniquement** sur le contexte fourni. Si la réponse ne s'y trouve pas, dis-le.

CONTEXTE :

{chunk_1}

{chunk_2}

{chunk_3}

QUESTION : {question_utilisateur}

RÉPONSE :

Le LLM reçoit le prompt augmenté et rédige la réponse.

Idéalement, il **cite ses sources** en utilisant les métadonnées des chunks.

À retenir

Le LLM **ne cherche pas**. C'est le retriever qui trouve les passages.

Le LLM ne fait que *rédiger* la réponse à partir du contexte.

Questions de réflexion

- Que se passe-t-il si le bon chunk est en position $k + 1$?

Composants techniques

1. **Document loaders**
2. **Text splitters / chunkers**
3. **Modèles d'embeddings**
4. **Vector stores**
5. **LLM** de génération
6. **Framework d'orchestration**

Vector stores : panorama

Vector store	Idéal pour
Chroma	Prototypage, local
FAISS	Recherche locale ultra-rapide
Pinecone	SaaS managé, production cloud
Weaviate	Recherche hybride, métadonnées riches
Qdrant	Open-source, écrit en Rust, self-hosted
Milvus	Très gros volumes, distribué
pgvector	Si PostgreSQL est déjà dans la stack
Elasticsearch	Existant Elastic, recherche hybride mature

- **LangChain** : leader, écosystème immense. Utilisé en TP.
- **LlamaIndex** : spécialisé RAG, fort sur les structures de données.
- **Haystack** : mature en production (deepset).
- **DSPy** : paradigme programmatique, optimisation auto des prompts.

Les types de RAG

- Naive RAG
- Advanced RAG
- Modular RAG
- Agentic RAG
- Self-RAG
- Corrective RAG (CRAG)
- GraphRAG
- HyDE
- Multi-modal RAG

C'est le RAG décrit jusqu'à présent : indexation simple, recherche par similarité, génération directe.

Limites :

- Sensible à la formulation de la question
- Pas de filtrage des chunks non pertinents
- Pas de reformulation, pas de raisonnement multi-étapes

Pré-retrieval

- *Query rewriting* : reformuler la question
- *Query expansion* : générer plusieurs variantes
- *HyDE* : embedder une *réponse hypothétique* plutôt que la question

Post-retrieval

- *Re-ranking* : cross-encoder pour réordonner précisément
- *Filtrage* : éliminer les chunks à faible score
- *Compression contextuelle* : ne garder que les passages utiles

Définition

Un *agent LLM* décide lui-même quand chercher, quoi chercher, dans quelle source, et boucle jusqu'à obtenir une réponse satisfaisante.

Il peut :

- interroger plusieurs sources (vector store, API, web, SQL)
- reformuler ses recherches
- décomposer la tâche
- s'auto-évaluer

- **Self-RAG** : le modèle est entraîné à émettre des *tokens spéciaux* pour décider quand récupérer, quand répondre, et s'auto-critiquer.
- **CRAG** (*Corrective RAG*) : un évaluateur juge la pertinence des documents récupérés. Selon le verdict (Correct/Ambigu/Incorrect) : réponse, re-recherche, ou recherche web.

- **GraphRAG** : la base n'est pas un sac de chunks, mais un *graphe de connaissances* (entités, relations). Idéal pour les questions multi-saut.
- **Multi-modal RAG** : étend le RAG aux images, tableaux, audio, vidéo, via des embeddings multimodaux (CLIP, modèles vision).

Tableau comparatif

Type	Apport	Complexité
Naive RAG	Base de référence	Faible
Advanced RAG	Reformulation, reranking	Moyenne
Modular RAG	Architecture flexible	Moyenne/Élevée
Agentic RAG	Décision autonome	Élevée
Self-RAG	Auto-évaluation	Élevée
CRAG	Correction par feedback	Élevée
GraphRAG	Relations multi-saut	Élevée
HyDE	Meilleur retrieval	Faible
Multi-modal RAG	Données non-textuelles	Moyenne/Élevée

À retenir

Commencez **toujours** par un Naive RAG. Mesurez. Puis évoluez.

Évaluation

Pourquoi évaluer ?

- Un RAG qui "marche bien" en démo peut être désastreux en prod
- Comparer des configurations (chunk 500 vs 1000, modèle A vs B)
- Détecter les régressions après mise à jour
- Construire la confiance des utilisateurs

À retenir

On n'optimise que ce qu'on mesure.

Définition

Jeu de questions de référence, avec réponses attendues et idéalement les documents qui contiennent ces réponses.

Construit :

- manuellement par des experts métier (plus fiable)
- ou généré par un LLM puis validé humainement (plus rapide)

- **Precision@k** : parmi les k chunks, combien sont pertinents ?
- **Recall@k** : combien des chunks pertinents existants sont récupérés ?
- **MRR** : position moyenne du premier chunk pertinent
- **NDCG** : ordre + pertinence graduée
- **Context Relevance (RAGAS)** : pertinence du contexte récupéré

- **Faithfulness / Groundedness** : la réponse est-elle fidèle au contexte ? (pas d'hallucination)
- **Answer Relevance** : répond-elle vraiment à la question ?
- **Answer Correctness** : factuellement correcte vs le gold ?
- **BLEU / ROUGE** : classiques, peu adaptées au RAG.

Framework	Spécificité
RAGAS	Le plus populaire, métriques RAG dédiées
TruLens	Observabilité + évaluation, dashboard
DeepEval	Style <i>pytest</i> pour LLM
LangSmith	Produit LangChain, intégré
Phoenix (Arize)	Observabilité open-source

En séance 3 : nous utiliserons **RAGAS**.

Avantages, limites, pièges

- Connaissance **à jour** (juste réindexer)
- **Traçabilité** des réponses
- Données **privées** préservées
- Coût **maîtrisé** vs fine-tuning
- **Réduction** des hallucinations
- **Flexibilité** : changer de LLM ou de vector store sans tout refaire

- Qualité **dépendante** du retrieval ("garbage in, garbage out")
- **Latence** accrue (2+ appels par requête)
- **Complexité** opérationnelle (pipeline, monitoring)
- Coûts **infra** (vector store, embeddings, observabilité)
- Mauvais sur les questions **transversales** (résumé global) — préférer GraphRAG

Attention

- Chunks trop grands ou trop petits
- Pas de chunk overlap
- Mauvais modèle d'embedding (anglais-only sur du français)
- Pas de métadonnées (pas de filtrage par date/auteur)
- Prompt système laxiste (le LLM répond "de mémoire")
- Top-k trop bas (le bon chunk est en position $k + 1$)
- Pas d'évaluation
- Données sales (obsolètes, dupliquées, contradictoires)

Cas d'usage

Où le RAG est-il utilisé en production ?

- Assistants documentaires d'entreprise
- Support client
- Assistants juridiques
- Assistants médicaux
- Assistants techniques / DevOps
- Tuteurs éducatifs
- Veille & recherche académique
- Chatbots produit

Écosystème & outils

- Abstractions unifiées (loaders, splitters, embeddings, vector stores, LLM)
- Chaînes prédéfinies (RetrievalQA, etc.)
- LangGraph pour les workflows complexes
- LangSmith pour le monitoring & l'évaluation

Avantages : maturité, écosystème immense.

Limites : API qui évolue vite, abstractions parfois lourdes.

LLM cloud : OpenAI, Anthropic, Google, Mistral, Cohere, AWS Bedrock

LLM open-weights : Llama, Mistral, Qwen, Gemma (Ollama, vLLM)

Embeddings cloud : OpenAI, Cohere, Voyage AI

Embeddings open-source : Sentence-Transformers, BGE, E5

Bridge vers la pratique

Nous construirons un **RAG simple mais propre** de bout en bout.

- Environnement Python
- Chargement de documents (PDF/Markdown)
- Chunking et embeddings
- Indexation dans un vector store local
- Première chaîne RAG fonctionnelle
- Premiers tests sur des questions concrètes

Nous itérons pour rendre le RAG robuste.

- Évaluation avec RAGAS
- Comparaison de stratégies de chunking
- Techniques avancées (re-ranking, query rewriting)
- Interface utilisateur (Streamlit ou Gradio)
- Discussion sur la mise en production

1. Les LLM sont puissants mais ont des **limites structurelles**
2. Le RAG est la réponse pragmatique : *examen à livre ouvert*
3. Deux phases : **indexation** (offline) + **retrieval** + **génération** (online)
4. Les composants : loader, chunker, embedder, vector store, LLM, orchestrateur
5. Plusieurs *types* de RAG : commencer simple, évoluer si besoin
6. L'**évaluation** systématique est indispensable

Questions ?

À la prochaine séance pour la pratique !